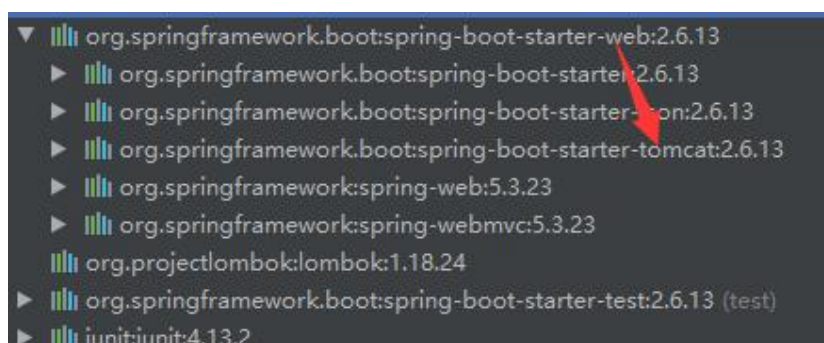


1. SpringBoot 设计目的是用来`简化 Spring 应用的 初始搭建以及开发过程`，包括 Spring+SpringMVC
2. SpringBoot 核心：所有子包之外必须包含一个入口类，resource 文件夹下必须有 application 命名的配置文件（YML 或 properties），名字必须为 application
3. 入口类 main 函数的作用：声明主应用类，加载注解及相应配置，启动 tomcat 容器，启动项目，并可以通过 args 加载一些临时外部参数。
4. SpringBoot 通过 starter 类的 jar 包来自动对两个组件进行配置
如 spring-boot-starter-jdbc, mybatis-spring-boot-starter 等，省去了繁杂的配置

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.mybatis.spring.boot</groupId>
  <artifactId>mybatis-spring-boot-starter</artifactId>
  <version>2.1.4</version>
</dependency>
```

Starter 的 jar 包会启动内置依赖，如：spring-boot-starter-web 的 jar，会启动内置的 tomcat



5. SpringBoot 的自动装配
 - (1) 什么是自动装配
在 SpringBoot 项目中，启动类只会扫描同级目录或者同级目录下的子目录中的 Bean，并注入到 IOC 容器中。
对于加载 JAR 包中的 Bean,它们的全限定名不在启动类的同级目录下，且全限定名未知。
在 SpringMVC 开发时，需要在 XML 中使用 Import 标签来手动引入 JAR 包中的 Bean。

```
<!-- 1. 开启缓存注解驱动 (AOP开关) -->
<cache:annotation-driven />

<!-- 2. 引入其他模块 -->
<import resource="classpath:redis.xml"/>
<import resource="classpath:cache-manager.xml"/>
<import resource="classpath:app-context.xml"/>

@SpringBootApplication
@EnableCaching
public class LeetcodeDemo {

    public static void main(String[] args) {
        // ...
    }
}
```

SpringBoot 则提供了自动加载机制，无需手动添加任何配置，即可完成对 JAR 包中 Bean 的加载；

如：Resttemplate、RedisTemplate 等，只需要引入 JAR 包，就能使用。

(2) 自动装配原理

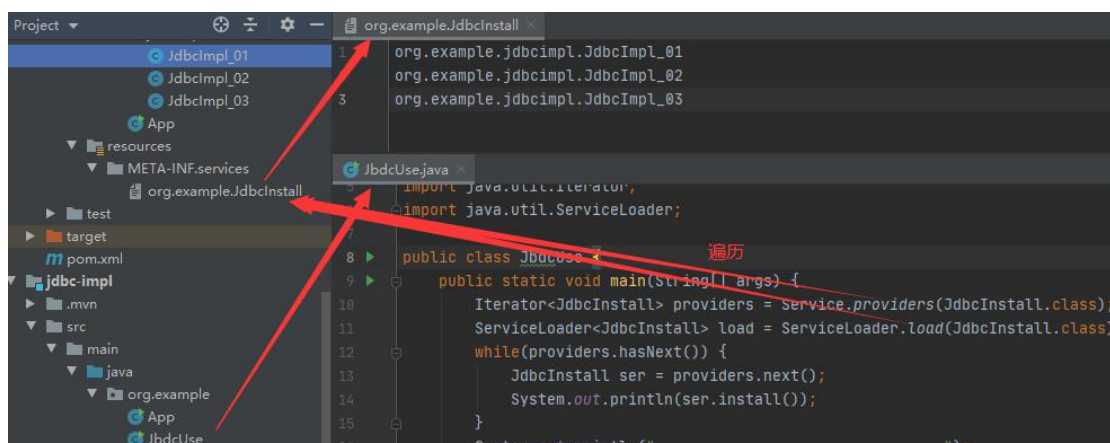
① JDK 的 SPI 机制

1) 概念

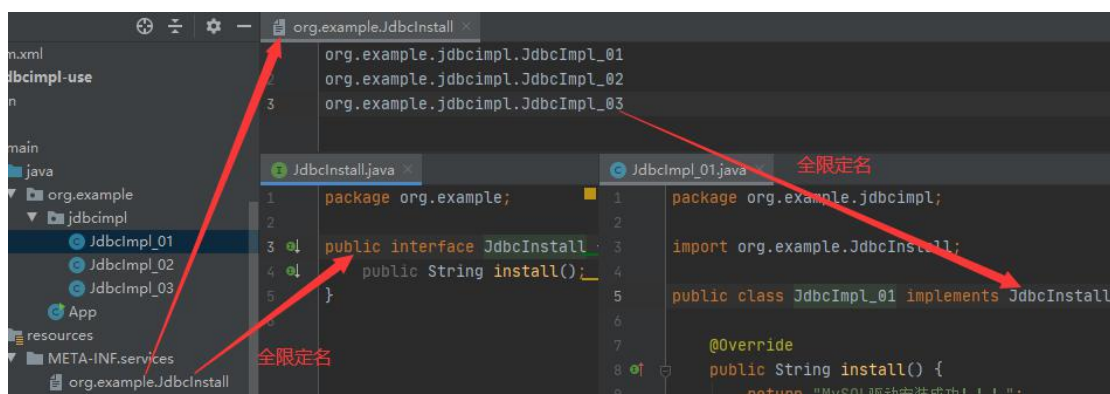
JDK SPI 是一种基于 ServiceLoader 的接口实现发现机制，它通过扫描 Jar 包中 META-INF/services 目录下的配置文件，动态加载接口实现类。

2) 具体实现方式

JDK 提供的 Service 类的 providers 方法或者 ServiceLoader 类的 load 方法，根据指定接口，遍历并发现，加载的 JAR 包中，resources 目录下的 META-INF/services 文件夹中该接口的实现类信息；



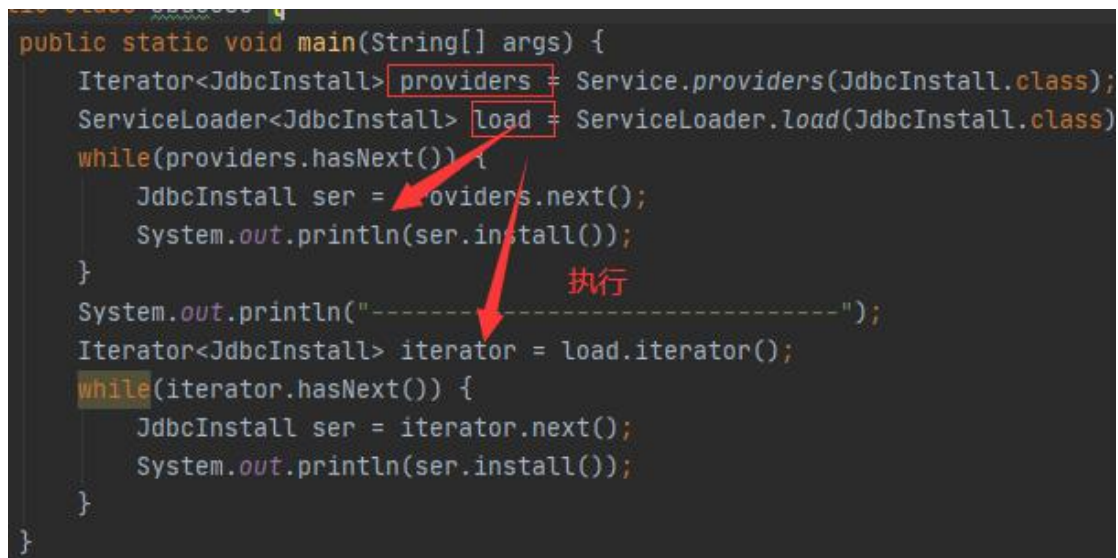
在该文件夹下，存在文件，名称为接口的全限定名，内容为该接口实现类的全限定名，可以是多个；



在根据接口全限定名找到该接口的多个实现类全限定名后，将实现类加载到 JVM 中，

可以使用迭代器等方式，遍历并执行接口的多种实现方式。

```
public static void main(String[] args) {
    Iterator<JdbcInstall> providers = Service.providers(JdbcInstall.class);
    ServiceLoader<JdbcInstall> load = ServiceLoader.load(JdbcInstall.class);
    while(providers.hasNext()) {
        JdbcInstall ser = providers.next();
        System.out.println(ser.install());
    }
    System.out.println("-----");
    Iterator<JdbcInstall> iterator = load.iterator();
    while(iterator.hasNext()) {
        JdbcInstall ser = iterator.next();
        System.out.println(ser.install());
    }
}
```



3) 应用

该技术特点是，强解耦，热插拔的特点，广泛应用于 java 项目的 JDBC 连接等；

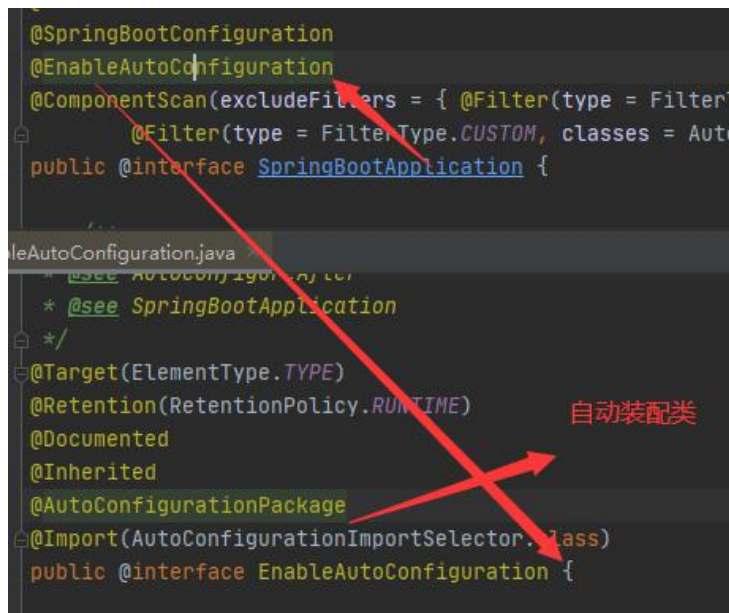
如 java 项目在数据库连接方面的处理：

首先，JDK 会内置 JDBC 接口规范，并在 JDK 启动时，会扫描加载的 JAR 包中，是否有该接口的实现，如果有则逐个建立连接（如：MySQL、oracle 等）。

② Spring boot 自动装配

Spring boot 的自动装配基于 JDK 的 SPI 机制开发的。

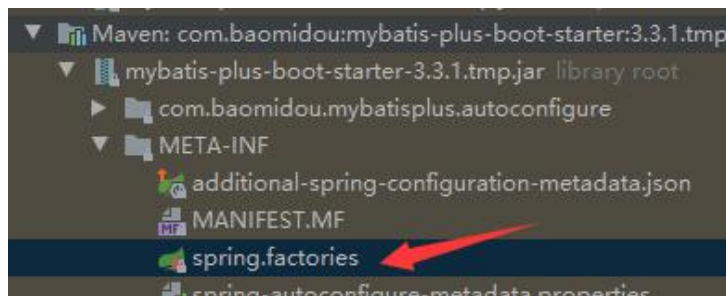
Springboot 的启动类注解 `@SpringBootApplication`，底层封装着 `@EnableAutoConfiguration` 注解，`@EnableAutoConfiguration` 注解底层又通过 `@Import` 注解关联着自动装配类。



```
@SpringBootApplication
@EnableAutoConfiguration
@ComponentScan(excludeFilters = { @Filter(type = FilterType.CUSTOM, classes = AutoConfigurationExcludeFilter.class) })
public @interface SpringBootApplication {
    // ...
}

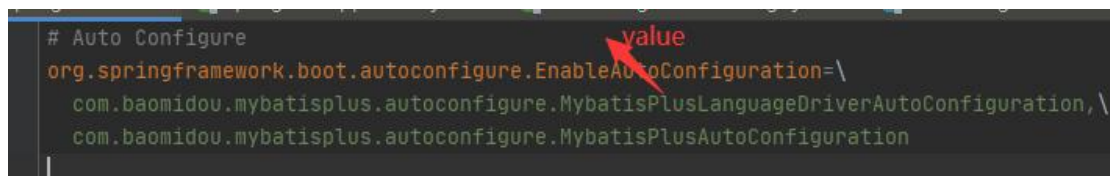
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@AutoConfigurationPackage
@Import({AutoConfigurationImportSelector.class})
public @interface EnableAutoConfiguration {
    // ...
}
```

该类实现了 `ImportSelector` 接口，该接口会扫描加载的 JAR 包中，META-INF 目录下的 `Spring.factories` 文件；



Spring.factories 文件主要负责 JAR 包中 Bean 的自动装配，还负责错误分析等功能；

Spring.factories 结构为 Key-values 的形式，key 为自动装配等功能的代码，Values 为对应类的全限定名



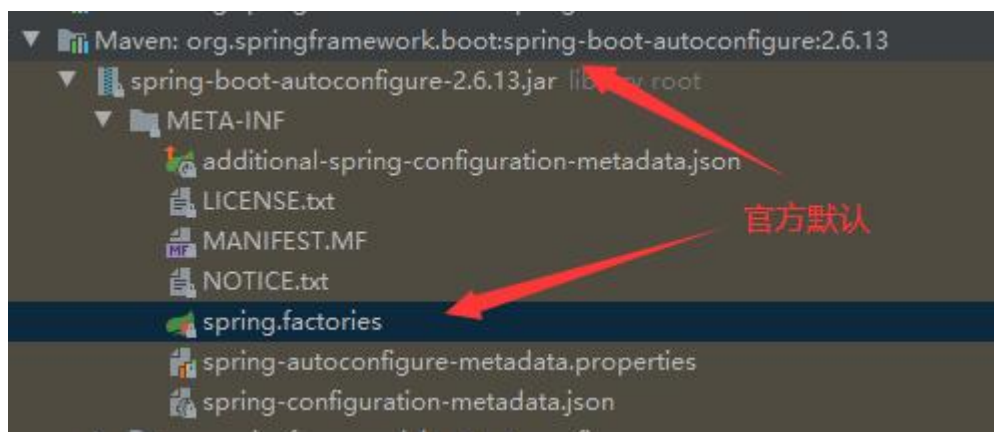
启动类启动时，会使用启动类注解底层封装的自动配置类，扫描每个 JAR 包中的 META-INF 目录中的 Spring.factories 文件，获取需要自动装配类的全限定名，然后根据全限定名将 Bean 注入到 IOC 容器中；

③ 官方包和非官方包

SpringBoot 的官方启动包以 spring-boot-starter-xxx 命名，通常不会包含 Spring.factories 文件

SpringBoot 默认会将官方包的自动装配类全部写在 SpringBoot 自带的 Spring.factories 文件中，比如 reids, rabbitmq 等

如果根据全限定名找到了对应的类就会自动装配该 Bean；



SpringBoot 的非官方包通常以 xxx-spring-boot-starter 命名，需要自带 Spring.factories 文件，SpringBoot 项目启动后会被扫描加载到 IOC 中；

6. SpringBoot 将对象交给 spring 管理，@configuration 类似于 @component (bean 标签)，使用方法有两种：
 - a) 在类之前使用 @configuration 交给 spring 管理，然后用 @autowire 去调用
 - b) 在类之前使用 @configuration，然后使用 @bean, @Bean 会将方法的返回值的类交给 Spring 管理，然后用 @autowire 去调用管理的类。这样的话就可以管理多个对象，且默认单例创建。

```

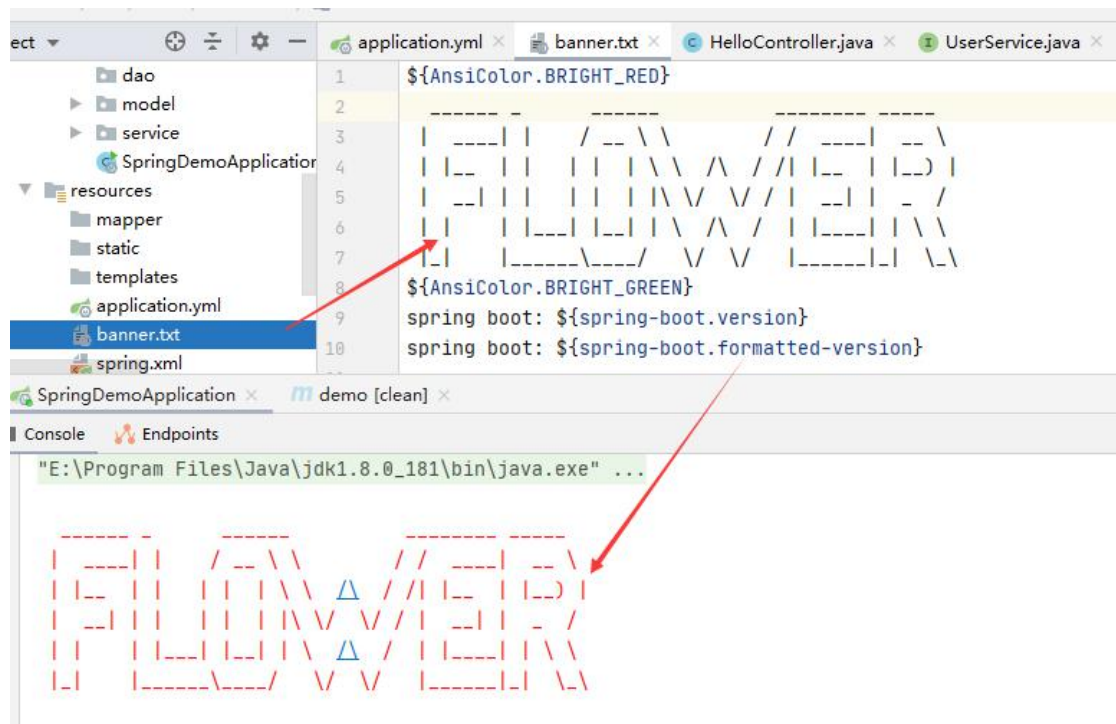
@Configuration
public class BeanConfigs {

    @Bean// (name = "aaa") //用来将方法返回值交给springboot管理 在工厂中默认
    @Scope("prototype") //prototype singleton
    public Calendar getCalendar() { return Calendar.getInstance(); }
}

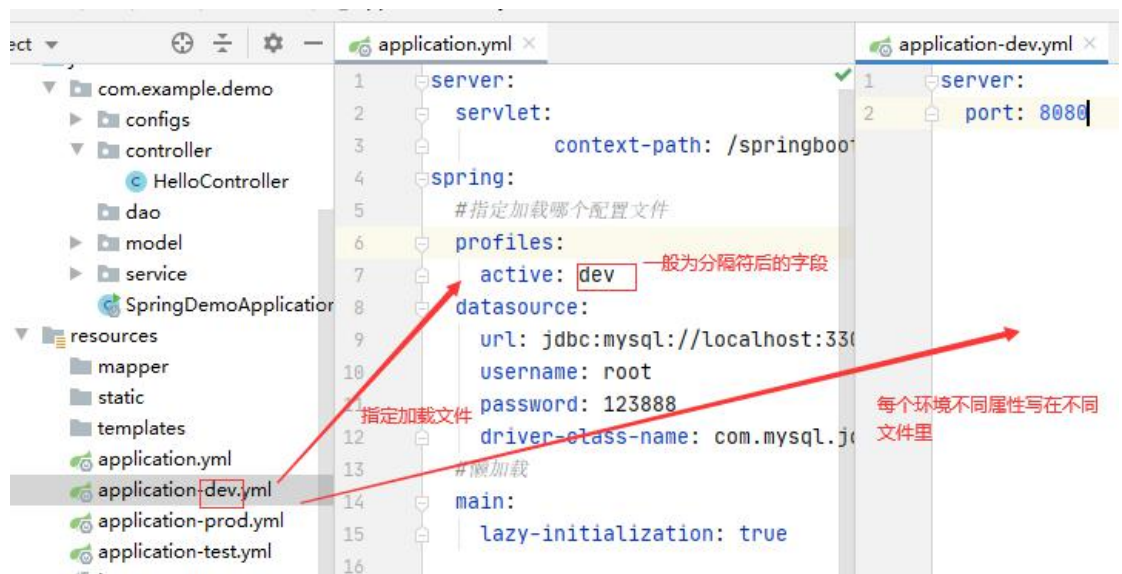
@Autowired
private Calendar calendar;

```

7. SpringBoot 控制台启动图标，在 resource 文件夹下添加 banner.txt 文件，启动的时候就会显示文件里的内容。

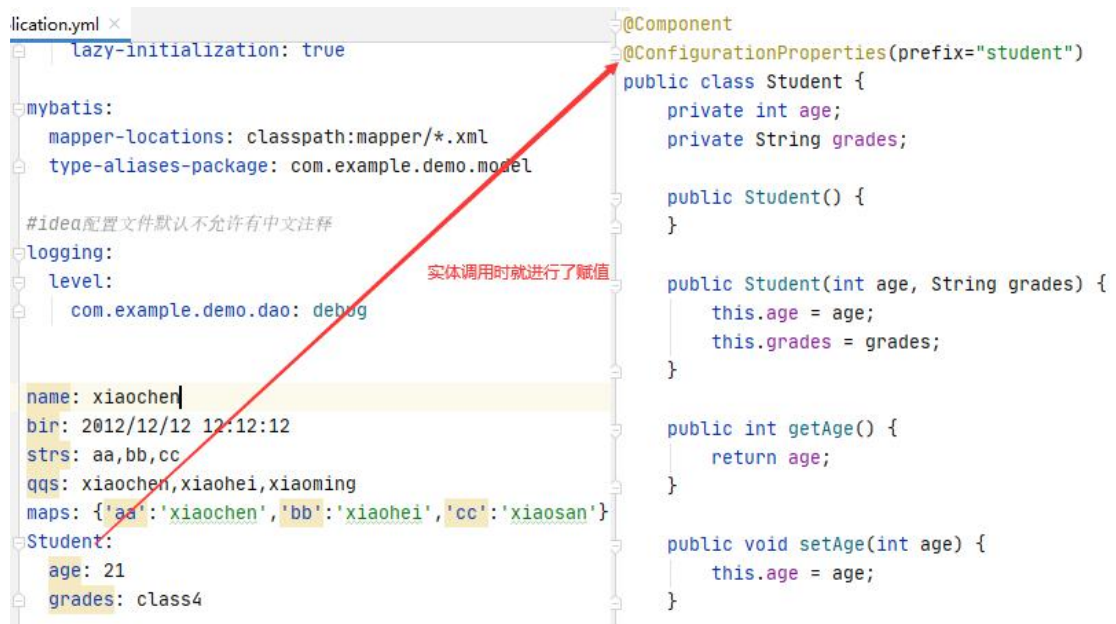


8. 开发、测试、生产不同配置文件的加载
- 首先有个公共的 application 配置文件，记录公共属性，并使用 spring.profiles.active 来决定激活哪个配置文件。
 - 创建不同环境下不同属性的配置文件，一般格式 spring-*, 如 spring-dev (开发环境), spring-prod (测试环境) 等,



9. 配置文件中变量的注入, `@value` 或 `@ConfigurationProperties`。





10. Spring 版本的特性

常用的 Spring 版本为：3.X 和 4.X，它们具有如下特性：

(1) 3.X

① 去 XML 化

3.X 主要做了去 XML 化，在之前的版本，Bean 等的创建主要依赖在 XML 中写 Bean 标签，在 3.X 版本使用注解的方式代替 Bean 等的创建；

② 模块驱动

模块驱动也是去 XML 化的重要体现，在 3.X 之前的版本中，要引入 Mybatis、Redis 等插件，需要在 XML 文件中使用 Import 标签来装载它们的配置文件，用到的 Bean 等；

Spring 3.X，可以使用 @EnableXXX 来开启相关机制并加载配置文件和相关的 Bean（如 Redis 使用 @EnableCache），或者使用 Jar 包提供的注解来加载配置文件和相关的 Bean（如 Mybatis 提供 @MapperScan）



(2) 4.X

4.X 版本则提供了 @Conditional 注解，可以有条件化的去实例化一个 Bean

springboot条件注解

@ConditionalOnBean (仅仅在当前上下文中存在某个对象时，才会实例化一个Bean)

@ConditionalOnClass (某个class位于类路径上，才会实例化一个Bean)

@ConditionalOnExpression (当表达式为true的时候，才会实例化一个Bean)

@ConditionalOnMissingBean (仅仅在当前上下文中不存在某个对象时，才会实例化一个Bean)

@ConditionalOnMissingClass (某个class类路径上不存在的时候，才会实例化一个Bean)

@ConditionalOnNotWebApplication (不是web应用)